

목차

- 이 문서를 읽는 법
- 1. 한눈에 보는 변화
- 2. 정상 워크로드 데이터셋
 - 왜 필요했나
 - 무엇을 넣었나
 - 어떻게 실행되나
 - 안전장치
 - 어느 규칙에 몇 개씩 걸려 있나
- 3. 규칙 자체를 고친 것
 - 3.1 컨테이너 정찰 규칙이 df 를 잡았다
 - 3.2 메모리 주입 규칙이 /proc 아래 쓰기를 전부 잡았다
 - 3.3 파일 없는 실행 규칙은 처음부터 안 잡히고 있었다
 - 3.4 커널 안에서 표현 못 하는 조건을 처리하는 자리
 - 3.5 차단 규칙
- 4. 측정이 실제보다 좋아 보이던 것
 - 4.1 아는 오탐을 모르는 칸으로 옮기고 있었다
 - 4.2 재현율과 정밀도가 서로 다른 것을 세고 있었다
 - 4.3 규칙별 점수판이 라운드를 안 갈랐다
 - 4.4 고치다가 실험 장비를 섞었다
 - 4.5 고친 뒤 숫자
- 5. 공격 체인
- 6. 검사로 남긴 것
- 7. 팀 전체 파이프라인은 지금 어디까지 도나
 - 7.1 설계상 6단계와 실제
 - 7.2 공격 생성
 - 7.3 검수
 - 7.4 실행
 - 7.5 탐지
 - 7.6 채점 경로가 둘이다
 - 7.7 되먹임
 - 7.8 오케스트레이션
 - 7.9 인프라
- 8. 이름만 있고 안 도는 것
- 9. 남은 일
 - 내 영역
 - 팀 차원에서 정해야 할 것

PurpleBPF 변경사항과 전체 흐름

앞선 설명서(8월 7일) 이후 바뀐 것과, 팀 전체 파이프라인이 지금 어디까지 도는지

이 문서를 읽는 법

앞 문서는 탐지 규칙이 무엇이고 어떻게 쓰는지를 다뤘다. 이 문서는 그 뒤로 아흐레 동안 바뀐 것을 다룬다. 앞 문서를 안 읽었어도 읽히도록 필요한 용어는 그 자리에서 다시 본다.

크게 두 부분이다. 앞쪽은 탐지 규칙 담당 영역에서 바뀐 것이고, 뒤쪽은 팀 전체 파이프라인이 지금 어떤 상태인지다.

가장 중요한 것부터 적는다. 앞 문서에서 "재현율 100%" 라고 적은 숫자는 지금 기준으로 보면 반쪽이었다. 공격을 쏘서 몇 개 잡았는지는 했는데, 정상 동작에서 규칙이 잘못 뜨는지는 안 됐다. 규칙을 넓히면 재현율은 무조건 오른다. 그 대가를 같이 재지 않으면 루프가 규칙을 계속 넓히는 쪽으로만 간다. 그 대가를 재는 것이 이번 작업의 절반이다.

나머지 절반은 그렇게 만든 측정 자체가 틀렸던 것을 찾아 고친 일이다.

1. 한눈에 보는 변화

	8월 7일	지금
관측 규칙	15	15
차단 규칙	1	8
실험용 대조군	4	4
정책 없이 판정하는 규칙	0	3
공격 체인	6	12
정상 워크로드 시나리오	0	74
재현율	100%	100%
정밀도	못 냄	84.6%
F1	못 냄	91.7%

관측 규칙 수가 그대로인 것은 우연이다. 하나를 지우고 하나를 새로 만들었다. 자세한 것은 3.4에 있다.

숫자만 보면 정밀도가 84.6% 로 100% 보다 낮아 보인다. 낮아진 것이 아니라 이제야 잴 수 있게 된 것이다.

2. 정상 워크로드 데이터셋

왜 필요했나

탐지 규칙의 성능은 두 숫자로 갈린다.

재현율	쓴 공격 중 몇 개를 잡았나	공격을 실행해야 나온다
정밀도	뜬 경보 중 몇 개가 진짜였나	정상 동작을 실행해야 나온다

앞 문서 시점에는 공격만 실행했다. 그래서 재현율만 나왔다. 이 상태의 위험은 이렇다. 규칙을 무한히 넓히면, 예를 들어 "파일을 여는 모든 동작을 잡는다" 로 쓰면 재현율은 100% 가 된다. 모든 공격이 파일을 열기 때문이다. 그런데 그 규칙은 쓸모가 없다. 정상 동작도 전부 잡기 때문이다.

폐쇄 루프는 미탐 목록을 다음 라운드 목표로 되먹인다. 재현율만 재면 루프는 미탐을 줄이는 쪽으로만 돌고, 규칙은 계속 넓어진다. 대가를 재지 않으면 루프가 잘못된 방향으로 수렴한다.

앞 문서 5.6의 2순위가 이 항목이었다. 그때 적어둔 방법은 "공격을 하나도 실행하지 않는 조용한 구간을 일정 시간 돌려 그때 뜨는 이벤트를 프로파일로 남긴다" 였다. 실제로는 그 방법을 안 썼다. 조용한 구간은 아무 일도 안 일어나는 구간이라 규칙이 안 뜨는 게 당연하다. 그건 정밀도가 아니라 배경 소음을 켜는 것이다.

대신 정상 동작을 일부러 실행하기로 했다. 공격을 실행해서 재현율을 재는 것과 정확히 대칭이다.

무엇을 넣었나

시나리오 74개다. 두 종류로 나눈다.

normal	23개	평범한 정상 동작. 어느 규칙도 안 건드리는 것이 정상
trap	51개	공격과 겉모습이 비슷하지만 정상인 것

trap 이 핵심이다. 평범한 동작만 넣으면 규칙이 안 뜨는 게 당연해서 아무것도 시험하지 못한다. 규칙이 아슬아슬하게 안 걸려야 하는 지점을 일부러 만들어야 규칙이 제대로 좁혀졌는지 알 수 있다.

예를 들면 이렇다. setuid 비트 설정을 잡는 규칙이 있다. setuid 비트는 파일에 붙이면 그 파일을 실행할 때 파일 주인의 권한을 얻는 표시다. 공격자가 권한을 올릴 때 쓴다.

chmod 4755 /tmp/x	규칙이 떴야 한다. setuid 비트를 세운다
chmod 755 /tmp/x	규칙이 뜨면 안 된다. 평범한 권한 변경이다
chmod 0755 /tmp/x	규칙이 뜨면 안 된다. 앞에 0 이 붙었을 뿐 같은 값이다

세 번째가 trap 이다. 8진수 표기가 헛갈리게 생겼을 뿐 setuid 비트는 안 붙는다. 규칙이 문자열을 보고 판단하면 여기서 걸리고, 비트를 보고 판단하면 안 걸린다.

시나리오 하나는 이렇게 생겼다.

```
{
  "label": "chmod_octal_lookalike",
  "kind": "trap",
  "goal": "8진수 표기가 setuid 처럼 보이지만 실제로는 비트가 안 붙는 권한 변경",
  "expect_silent": ["t1548-001-setuid-bit-set"],
  "steps": [
    { "order": 1, "purpose": "앞에 0 이 붙은 8진수로 권한을 준다",
      "command": "cp /bin/true /tmp/x && chmod 0755 /tmp/x && stat -c %A /tmp/x" }
  ]
}
```

expect_silent 가 "이 워크로드에서 이 규칙은 뜨면 안 된다" 는 선언이다. 이게 있어야 자동 채점이 된다.

어떻게 실행되나

공격 실행기와 같은 컨테이너 설정으로 돌린다. 비교가 성립하려면 조건이 같아야 하기 때문이다.

privileged 없음	호스트와 맞먹는 권한을 안 준다
capability 추가 없음	관리자 권한을 쪼갬 단위를 안 더 준다
no-new-privileges	setuid 로 권한이 올라가는 것을 커널이 막는다
호스트 마운트 없음	호스트 디스크를 안 보여준다
도커 기본 seccomp	부를 수 있는 시스템콜 목록이 기본 그대로다

공격 쪽과 다른 점은 둘이다. 기록하는 테이블이 다르고(benign_log), 명령이 실패해도 멈추지 않는다. 공격은 실패하면 측정이 안 되지만, 정상 워크로드는 명령이 실패해도 그 사이에 규칙이 났는지가 여전히 의미 있다.

안전장치

시나리오가 자동 생성이라 사람이 다 읽지 않는다. 그래서 실행 전에 기계로 두 번 거른다.

첫째, 금지 항목 검사(tests/test_benign_safety.py)다. 컨테이너 안이라도 건드리면 안 되는 것들을 정규식으로 잡는다.

런타임 소켓 경로	/var/run/docker.sock 등
setuid 비트 설정	chmod 4755, chmod u+s
외부 네트워크 접속	curl, wget 으로 바깥에 나가는 것
클라우드 메타데이터 접속	169.254.169.254 로 실제 붙는 것
시스템 계정 파일 덮어쓰기	/etc/passwd, /etc/shadow, /etc/sudoers
디스크 조작	mkfs, fdisk, dd of=/dev/sd*

여기에 expect_silent 에 적힌 규칙 이름이 실제로 있는 이름인지도 본다. 처음에는 규칙 이름 목록을 검사 파일 안에 적어뒀는데, 규칙을 늘렸을 때 같이 안 고쳐서 어긋났다. 그래서 규칙 파일에서 직접 읽게 바꿨다.

둘째, 실행 검사(tests/smoke_benign.py)다. 금지 항목 검사는 쓰면 안 되는 것을 안 썼지만 본다. 명령이 문법적으로 깨졌는지, 없는 프로그램을 불렀는지는 실제로 돌려봐야 안다.

이걸 만든 이유가 있다. 시나리오 절반쯤이 파이썬으로 서버나 워커를 띄우게 짜여 있었는데, 실행 이미지인 ubuntu:22.04 에 파이썬이 없었다. 아홉 개가 통째로 실패했다. perl 로 다시 쓰면서 또 걸렸다. perl 은 있는데 최소 설치판이라 표준 라이브러리 일부가 없다.

python3 없음	9개 시나리오를 perl 로 다시 씀
PerlIO 없음	메모리 안 파일핸들을 파이프로 바꿈
Digest::SHA 없음	해시 대조를 내용 대조로 바꿈

지금은 74개 전부 컨테이너에서 실제로 돌아가는 것을 확인했다.

어느 규칙에 몇 개씩 걸려 있나

expect_silent 에 그 규칙이 적힌 시나리오 수다. 그 규칙에 대해 오탐 시험이 몇 번 일어나는지를 뜻한다.

규칙	시나리오
t1105-tmp-exec	47
t1548-001-setuid-bit-set	21
exec-admin-tools	20
t1562-001-defense-tamper	18
t1613-container-discovery	18
t1055-009-proc-mem-inject	13
fileless-fd-exec	12
t1055-008-pttrace-inject	11
t1548-003-sudo-abuse	8
t1552-001-cred-file-read	8
t1611-host-mount	8
exec-file-capability	8
t1611-namespace-change	7
t1552-005-cloud-metadata	6
t1610-runtime-socket-connect	5

쏠려 있다. t1105 는 임시 경로에서 무언가를 실행하는 것을 잡는 규칙인데, 임시 파일을 쓰는 정상 동작이 워낙 많아서 자연히 많이 걸렸다. 아래쪽 규칙들은 시나리오가 적어서 오탐을 놓칠 수 있다. 남은 일에 적어됐다.

3. 규칙 자체를 고친 것

정상 워크로드를 넣자 규칙이 정상 동작에 뜨는 것이 실제로 보이기 시작했다. 그중 규칙 잘못된 것을 고쳤다.

3.1 컨테이너 정찰 규칙이 df 를 잡았다

t1613-container-discovery 는 컨테이너 안에 있는지, 무엇이 붙어 있는지 확인하는 정찰 동작을 잡는다. 공격자가 발판을 잡은 직후 가장 먼저 하는 일이다.

원래 조건은 경로가 /cgroup 이나 /mountinfo 로 끝나는 파일을 여는 것이었다. cgroup 은 리눅스가 프로세스에 자원 한도를 매기는 장치이고, mountinfo 는 무엇이 마운트돼 있는지 적힌 파일이다. 둘 다 자기가 컨테이너인지 알아내는 데 쓰인다.

두 군데서 잘못 났다.

ls /sys/fs/cgroup	컨테이너가 자기 메모리 한도를 읽으려고 여는 디렉터리
	경로가 /cgroup 으로 끝나서 걸렸다
df -k /	남은 디스크 용량을 계산하려면 마운트 목록이 필요하다
	/proc/self/mountinfo 를 읽는다. 정찰과 여는 파일이 같다

첫째는 조건을 좁혀 해결했다. /proc/ 아래로 한정하면 /sys/fs/cgroup 은 빠진다. 한 selector 안에 조건을 두 개 넣으면 둘 다 만족해야 걸린다.

둘째는 경로만으로 못 가른다. 정찰이 여는 파일과 df 가 여는 파일이 글자 그대로 같다. 그래서 마운트 목록이 일인 프로그램들을 실행 파일 이름으로 뺐다.

```
matchBinaries:
- operator: "NotIn"
  values: [ "/usr/bin/df", "/bin/df", "/usr/bin/mount", ... ]
```

깨끗한 해결이 아니다. 공격자가 자기 도구를 /usr/bin/df 로 두면 빠져나간다. 그래도 남긴 이유는 이 규칙 하나로 막는 게 아니기 때문이다. 이름을 바꿔 심는 행위 자체가 다른 규칙에 걸린다. 이 한계는 규칙 주석에 적어뒀다.

실측으로 확인했다.

df -k /	통과	
ls /sys/fs/cgroup	통과	
/usr/bin/df-fake 로 읽기	뚫	이름만 비슷한 것은 안 빠진다
cat /proc/self/mountinfo	뚫	
cat /proc/self/cgroup	뚫	

3.2 메모리 주입 규칙이 /proc 아래 쓰기를 전부 잡았다

t1055-009-proc-mem-inject 는 남의 프로세스 메모리를 고치는 것을 잡는다. /proc/<번호>/mem 은 그 프로세스의 메모리를 파일처럼 열어놓은 것이라 여기에 쓰면 메모리가 바뀐다.

원래 조건은 /proc/ 로 시작하는 경로에 쓰기였다. 그런데 /proc 아래에는 메모리 말고도 온갖 것이 있다.

```
echo 200 > /proc/self/oom_score_adj    메모리 부족 때 죽는 순서를 조정한다
echo worker > /proc/self/comm           프로세스 이름을 적는다
```

둘 다 평범한 동작인데 규칙에 걸렸다. 경로가 /mem 으로 끝나는 것으로 한정했다. 스레드까지 치면 /proc/<번호>/task/<번호>/mem 인데 이것도 /mem 으로 끝난다.

3.3 파일 없는 실행 규칙은 처음부터 안 잡히고 있었다

이게 이번에 찾은 것 중 제일 크다.

T1620 은 디스크에 파일을 안 남기고 코드를 실행하는 기법이다. memfd 라는 것으로 이름만 있고 파일시스템에는 없는 파일을 만들고, 거기에 페이로드를 넣어 실행한다. 디스크를 아무리 뒤져도 흔적이 없다.

규칙은 두 혹을 걸고 있었다.

```
sys_memfd_create    익명 메모리 파일을 만드는 시스템콜. 조건 없음
security_bprm_check 실행 직전 지점. 경로가 memfd: 로 시작하면
```

두 문제가 겹쳐 있었다.

첫째, 만드는 것과 실행하는 것이 한 규칙에 묶여 있었다. 조건 없이 sys_memfd_create 를 잡으면서 그걸 T1620 에 매핑했으므로, 만들기만 하고 실행이 실패해도 잡은 것으로 세어진다. 파일 없는 실행이 안 일어났는데 기록은 남는다.

둘을 갈라놓고 다시 돌리자 T1620 이 미탐으로 바뀌었다. 그동안 잡히던 것은 전부 만드는 쪽에서 온 거였고, 실행 쪽은 한 번도 안 잡히고 있었다.

둘째, 실행 쪽이 왜 안 잡히는지 확인했다. 혹에 조건을 안 걸고 무엇이 오는지 그대로 찍어봤다.

```
{"linux_binprm_arg": {"path": "/usr/bin/cat", "permission": "-rwxr-xr-x"}}
{"linux_binprm_arg": {"path": "/usr/bin/perl", "permission": "-rwxr-xr-x"}}
{"linux_binprm_arg": {"path": "/usr/bin/perl", "permission": "-rwxrwxrwx"}}
```

세 번째가 memfd 실행이다. path 항목이 통째로 없다. 파일시스템에 경로가 없으니 커널이 줄 경로가 없다. 접두사로 맞출 대상 자체가 없었던 것이다. 진짜 실행 파일(ELF)을 넣어 다시 해봐도 같았다. memfd 에 붙인 이름은 그 파일의 다른 표기에만 있고 실행 지점에는 안 들어간다.

경로가 없다는 것 자체가 신호인데, 규칙 문법에 "비어 있다" 를 쓸 방법이 없다. 커널 안에서 표현이 안 되는 조건이다.

그래서 판정 위치를 옮겼다. 커널이 내보내는 실행 이벤트에는 그 실행을 어떻게 불렀는지가 남는다.

```
execveat(fd, "", AT_EMPTY_PATH) -> /dev/fd/<번호>
execve("/proc/self/fd/<번호>")   -> /proc/self/fd/<번호>
```

둘 다 커널이 붙이는 이름이다. 평범한 프로그램은 경로로 실행하지 파일 번호로 실행하지 않는다. 이 조건은 커널 밖에서 판정한다. 아래 3.5에서 다룬다.

원래 규칙 파일은 지웠다. 안 맞는 규칙을 남겨두면 룰팩이 커 보이기만 한다.

3.4 커널 안에서 표현 못 하는 조건을 처리하는 자리

앞 문서 5.6의 4순위에 "비정상 셸 실행" 이 있었다. 웹 서버나 데이터베이스가 자식으로 셸을 띄우는 것을 잡는 규칙인데, 부모 프로세스가 무엇이냐는 조건을 커널 안에서 표현할 수 없어 stream_rules 라는 자리에 명세만 적어뒀다.

그 자리는 문서에만 있고 구현이 없었다. 판정 코드가 없으니 규칙을 만들어도 무조건 미탐이 된다.

이번에 구현했다. 매핑 파일에 사람이 읽는 설명 말고 기계가 읽는 조건을 같이 적고, Mapper 가 실행 이벤트 스트림에서 판정한다.

```
anomalous-shell-spawn:
  technique: T1059.004
  match:
    kind: parent_child_binary
    child_postfix: [ "/sh", "/bash", "/dash", "/zsh", "/ash" ]
    parent_postfix: [ "/nginx", "/httpd", "/postgres", "/mysqld", ... ]
```

지금 세 종류가 있다.

규칙	기법	판정 방식
anomalous-shell-spawn	T1059.004	부모가 서버 프로세스이고 자식이 셸
fileless-fd-exec	T1620	실행 경로가 /dev/fd/ 나 /proc/self/fd/ 로 시작
setuid-exec-escalation	T1548.001	실행하면서 uid 와 euid 가 같림

조건만 적고 판정 방식(match)을 안 적은 항목은 경고를 내고 건너뛴다. 조용히 지나가면 규칙이 있는 줄 알고 미탐으로 오해하게 된다.

셋째 것은 이 환경에서 뜰 수가 없다. 이유는 4.3에 적었다.

3.5 차단 규칙

앞 문서 시점에는 차단 규칙이 하나뿐이었다. 지금은 여덟이다.

차단 규칙은 관측 규칙을 복사해 동작 지시만 붙이면 되는 것이 아니다. 어떤 방식으로 막아야 실제로 막히는지가 흑마다 다르고, 그것은 돌려봐야 안다. 흑별로 확인한 결과가 rules/tracingpolicies/enforce/README.md 에 있다.

이번에 두 가지 사고를 찾았다.

첫째, 관측판을 고치고 차단판을 안 고쳐서 둘이 달라져 있었다. t1613 을 좁혀 df 오탐을 없앴는데 차단판은 넓은 조건 그대로였다. 그 상태로 올리면 df 를 못 쓰게 만든다. 관측판으로 켜 정밀도가 차단판의 정밀도가 아니게 되는데, 오탐 한 건과 정상 동작이 막히는 것은 무게가 다르다.

둘째, 그 차단판은 selector 세 개 중 하나에만 동작 지시가 붙어 있었다. 나머지 둘은 관측만 하고 안 막는다. 파일이 enforce/ 아래 있으니 막히는 줄 알게 된다.

둘 다 검사로 남겼다(tests/test_enforce_sync.py). 조건이 다르거나, 거는 혹은 다르거나, 동작 지시가 빠진 selector 가 있으면 잡는다.

차단판을 올릴 때 새로 알게 된 위험도 적어졌다. t1613 차단판을 올려두면 docker run 이 아예 실패한다. 도커가 컨테이너를 만들면서 /.dockerenv 를 직접 만들고 여는데, 규칙이 그걸 정찰로 보고 막기 때문이다. 검증할 때는 컨테이너를 먼저 띄우고 그다음에 차단판을 올려야 한다.

4. 측정이 실제보다 좋아 보이던 것

여기부터가 이번 작업에서 제일 중요하다. 위의 것들을 다 하고 나서 검토를 돌렸더니, 만들어놓은 측정 자체가 결과를 좋게 보이게 하고 있었다.

4.1 아는 오탐을 모르는 칸으로 옮기고 있었다

판정은 이렇게 돼 있었다.

FP	expect_silent 에 적어둔 규칙이 떴다
UNEXPECTED	안 적어둔 규칙이 떴다
CLEAN	아무 규칙도 안 떴다

그리고 정밀도는 FP 만 썼다. 즉 뜯 걸 알면서 expect_silent 에 안 적기만 하면 정밀도가 안 떨어진다.

실제로 그 상태였다. 시나리오 셋이 규칙을 띄우는데, 설명에는 "이 워크로드에서는 이 규칙이 뜬다" 고 적어놓고 expect_silent 에서는 빼놨다. 그래서 UNEXPECTED 로 분류되고 정밀도 분모 밖으로 나갔다. 그 라운드의 정밀도는 100% 로 나왔다.

정상 워크로드에서 규칙이 뜨면 그게 오탐이다. 미리 적어뒀는지는 오탐이냐 아니냐를 가르지 않는다. 그건 내가 예상했느냐일 뿐이다. 그래서 판정을 이렇게 바꿨다.

FP	규칙이 하나라도 떴다
CLEAN	아무 규칙도 안 떴다

예상했나	declared	쫓혔다고 생각한 규칙이 여전히 뜬다. 트랩이 제 일을 했다
	undeclared	건드릴 줄 몰랐던 규칙이 떴다. 규칙을 다시 봐야 한다

4.2 재현율과 정밀도가 서로 다른 것을 세고 있었다

재현율의 분자는 잡은 기법 수였다. 정밀도의 분모는 오탐이 난 시나리오 수였다.

$$\begin{aligned}\text{정밀도} &= \text{잡은기법} / (\text{잡은기법} + \text{오탐시나리오}) \\ &= 11 / (11 + 3)\end{aligned}$$

기법 11개와 시나리오 3개를 더한 값이다. 서로 다른 것을 더했으니 이 숫자는 아무것도 아니다.

양쪽을 실행 단위로 맞췄다. 공격을 한 번 실행한 것과 정상 워크로드를 한 번 실행한 것은 둘 다 "한 번 돌렸고 경보가 떴거나 안 떴다" 라서 같이 셀 수 있다.

기법 단위 커버리지를 없앤 건 아니다. 묻는 게 다르기 때문에 자리를 따로 뒀다. 어느 기법을 덮느냐와 뜯은 경보 중 진짜가 얼마냐는 다른 질문이다.

4.3 규칙별 점수판이 라운드를 안 갈랐다

라운드는 루프의 세대다. 규칙을 고치고 다시 돌리면 라운드가 하나 올라간다.

점수판이 라운드를 안 가르고 전체를 합산하고 있었다. 그런데 공격은 1라운드부터 계속 돌렸고 정상 워크로드는 최근 몇 라운드에만 있다. 그래서 잡은 건수는 처음부터 누적되는데 오탐은 최근 것만 세어졌다. 정밀도가 실제보다 높게 나온다.

라운드 사이에 규칙 파일이 바뀐다는 점까지 겹치면 더 나쁘다. 서로 다른 버전 규칙의 성적이 한 줄에 합산된다. 고친 규칙이 옛 오탐을 계속 지고 가고, 망가뜨린 규칙이 옛 성적에 가려진다.

라운드별로 가르고 화면에는 최신 라운드만 보여주게 바꿨다.

4.4 고치다가 실험 장비를 섞었다

기법 번호를 안 붙인 규칙이 셋 있다. `exec-admin-tools`(컨테이너 안에서 패키지 관리자나 네트워크 도구 실행), `exec-file-capability`(파일에 권한 부여), `memfd-create`(익명 메모리 파일 생성)다. 후속 행위에 따라 무슨 기법인지 갈리거나, 공격 탐지가 아니라 정책 위반 신호라서 일부러 안 붙였다.

Mapper 는 기법 번호가 없는 규칙의 이벤트를 통째로 버리고 있었다. 그래서 이 셋은 오탐을 쟈 수가 없었다. 정상 워크로드 25개가 이 규칙들을 `expect_silent` 에 걸어뒀는데, 위반될 수 없는 선언이라 "조용했다" 만 늘리고 있었다.

기법이 없다는 것과 안 떠도 된다는 것은 다르다. 그래서 기록하도록 바꿨다.

그런데 "기법 없으면 다 기록" 으로 바꾼 것이 지나쳤다. `experiments/` 에 있는 대조군 정책이 같이 딸려 들어왔다. 이건 `io_uring` 실험용으로 일부러 뚫리게 만든 장비라 규칙 품질과 무관하다. 정상 워크로드 20건이 이 장비의 발화로 오탐 판정을 받았다.

양쪽 다 명시로 받게 고쳤다. 매핑 파일에 `unmapped_policies` 항목을 만들어 "기법은 없지만 오탐은 재는 규칙" 을 적고, 거기에도 없고 기법 목록에도 없는 정책은 버린다. 모르는 정책의 이벤트가 오탐 데이터로 조용히 섞이면 안 된다.

오염된 탐지 기록 5317건을 지우고 다시 켜다.

4.5 고친 뒤 숫자

같은 조건으로 두 번 돌려 같은 값이 나오는 것을 확인했다.

라운드 27, 28

TP 11 공격 실행 11번 중 11번 탐지됨
FN 0 놓친 것 없음
INVALID 1 공격이 실행 자체가 안 됨 (아래 설명)
FP 2 정상 워크로드 74개 중 2개에서 규칙이 뜸
TN 72 나머지 72개는 조용했음

재현율 100.0 정밀도 84.6 F1 91.7
뺀 기법 11개

규칙별로 보면 이렇다.

규칙	잡음	오탐	정밀도
t1105-tmp-exec	1	1	50.0
t1562-001-defense-tamper	1	1	50.0
나머지 9개	1	0	100.0

남은 오탐 둘은 규칙 결함이 아니다. 정상 동작과 공격이 실제로 겹치는 지점이다.

/tmp 에서 실행 빌드가 하는 일이면서 공격이 하는 일이다
SIGTERM 전송 정상 종료 신호면서 감시 프로그램을 죽이는 방법이다

좁히려면 "컨테이너가 시작된 뒤에 만들어진 파일인가" 같은 조건이 필요한데, 지금 쪽으로는 표현이 안 된다.

INVALID 는 공격이 실행 자체에 실패한 경우다. T1611(컨테이너 탈출)이 여기 해당한다. 도커의 기본 seccomp 설정이 해당 시스템콜을 막아서 명령이 커널에 닿지도 못한다. 공격이 안 나갔으므로 탐지가 없는 것이 맞다. 이걸 미탐으로 세면 있지도 않은 구멍이 다음 라운드 목표로 되먹여져 루프가 제자리를 돈다.

같은 이유로 체인을 안 만든 규칙이 하나 더 있다. setuid-exec-escalation 은 setuid 로 권한이 실제로 올라가는 순간을 잡는데, 실행 컨테이너에 no-new-privileges 가 걸려 있어서 setuid 비트가 붙어 있어도 커널이 권한을 안 준다. 비특권 사용자로 내려가 실행해도 uid 와 euid 가 안 갈린다. 규칙은 지우지 않았다. 그 설정을 안 거는 환경에서는 그대로 쓴다.

5. 공격 체인

앞 문서 시점에는 씨앗 체인이 6종이었고, 나중에 규칙이 15종으로 늘면서 9종이 됐다. 그런데 규칙은 15종인데 체인은 9종이었다. 재현율 100% 는 "쏜 9개 중 100%" 지 룰팩 전체가 아니었다.

셋을 더해 12종이 됐다.

T1620 익명 메모리에 페이로드를 넣고 파일 번호로 실행
T1562.001 감시 프로세스에 SIGTERM 을 보내고 안 죽으면 SIGKILL
T1059.004 서버 프로세스가 자식으로 셸을 띄움

체인이 없는 규칙이 하나 남았다. t1610-runtime-socket-connect 는 컨테이너 런타임 소켓에 붙는 것을 잡는데, 그 소켓이 컨테이너 안에 마운트돼 있어야 시도라도 할 수 있다. 컨테이너가 애초에 취약하게 설정돼 있어야 하는 상황은 측정 대상에서 빼기로 팀에서 합의했다.

6. 검사로 남긴 것

이번에 찾은 문제 중 여럿이 "고쳐놓고 다른 데를 안 고쳐서 조용히 어긋난" 종류였다. 그런 건 다시 벌어지므로 검사로 막았다.

검사	무엇을 막나
tests/test_benign_safety.py	시나리오가 금지 동작을 하거나, 없는 규칙 이름을 쓰는 것
tests/smoke_benign.py	시나리오 명령이 실제로는 안 도는 것
tests/test_enforce_sync.py	차단판이 관측판과 조건이 다르거나 동작 지시가 빠진 것

셋 다 일부러 어긋뜨려서 실제로 잡히는지 확인했다.

7. 팀 전체 파이프라인은 지금 어디까지 도나

여기부터는 내 담당 영역 밖이다. 코드를 읽어 확인한 것만 적는다. "문서에 적혀 있다" 와 "코드에 있다" 와 "실제로 돈다" 를 구분해서 쓴다.

7.1 설계상 6단계와 실제

단계	하는 일	구현	자동으로 이어지나
1 생성	기법 번호를 받아 공격 명령 묶음을 만든다	있음	아니오
2 검수	그 명령이 안전하고 말이 되는지 본다	자동 검사 있음, 사람 승인 없음	아니오
3 실행	격리 컨테이너에서 돌리고 정답지를 남긴다	있음	예
4 탐지	커널 이벤트를 받아 답안을 남긴다	있음	예
5 채점	정답지와 답안을 맞춰 점수를 낸다	있음	예
6 되먹임	놓친 기법을 다음 라운드 목표로 넣는다	없음	아니오

3에서 5까지는 Dagster 라는 도구가 순서대로 이어준다. 1과 2는 그 그래프에 안 들어 있고, 6은 코드가 아예 없다.

7.2 공격 생성

Neo4j 라는 데이터베이스에 ATT&CK 기법 정보를 넣어두고, 대상 기법의 정보를 꺼내 gemma 라는 언어 모델에게 넘겨 공격 명령을 만들게 한다. Neo4j 는 점과 선으로 관계를 저장하는 데이터베이스다.

모델에게 넘기는 것은 넷이다. 기법의 이름과 설명 전문, 그 기법이 속한 전술 목록, 상위 기법과 하위 기법 목록, 그리고 출력 형식 예시다. 프롬프트에 "설명에 없는 도구나 명령을 지어내지 마라" 는 지시가 들어 있다.

지식그래프라고 부르지만 지금 들어 있는 관계는 두 종류뿐이다. 어느 전술에 속하는가와 어느 기법의 하위인가다. 공격의 선후관계, 즉 이걸 하면 저게 가능해진다는 연결은 아직 없다. 코드 주석이 그렇게 밝히고 있다. 그래서 지금은 기법 하나짜리 정보 카드를 모델에게 넘기는 것에 가깝다.

중요한 사실이 하나 있다. 지금까지 문서에 기록된 실측 숫자는 전부 손으로 쓴 씨앗 체인으로 나온 것이다. 생성 경로로 나온 게 아니다. 씨앗 체인은 규칙을 검증하려고 내가 무엇을 썼는지 확실히 아는 고정 체인이다. 생성기가 아직 학습할 것이 없는 0라운드에서 룰팩을 먼저 검증하려고 만들었다.

7.3 검수

자동 검사는 셋을 본다.

구조 기법 번호와 목표가 비었는지, 단계 번호가 1부터 연속인지
문법 shellcheck 로 셸 명령의 문법 오류와 위험한 표현을 찾는다
순서 먼저 와야 하는 단계가 앞에 있는지

셋을 통과하면 승인, 순서만 어긋나면 검토 필요, 구조나 문법이 깨지면 거부다.

주의할 점이 둘 있다. shellcheck 가 안 깔린 컴퓨터에서는 문법 검사를 건너뛰고 통과 처리한다. 그리고 순서 규칙은 지금 한 개뿐이다. T1611 에서 chroot 가 mount 보다 앞에 오면 안 된다는 것 하나다. 나머지 기법은 순서 검사를 무조건 통과한다.

사람 승인은 절반만 있다. Slack 으로 알림을 보내는 코드가 있고 메시지에 승인 버튼과 반려 버튼까지 그려진다. 그런데 그 버튼을 눌렀을 때 받는 서버가 없다. 그리고 알림을 보내는 함수를 파이프라인에서 부르는 곳도 없다. 실행기는 자동 검사 결과만 보고 바로 실행한다.

희수님이 만든 3단계 정적 검증기가 따로 있는데 파이프라인에 안 붙어 있다. 지금 상태로는 임포트 경로가 어긋나 실행 자체가 안 된다.

7.4 실행

ubuntu:22.04 컨테이너를 띄우고 단계마다 명령을 하나씩 돌린 뒤 컨테이너를 지운다. 안전 옵션은 2장에 적은 것과 같다. 호스트에서 공격 명령을 직접 돌리는 경로는 없다.

기록하는 것은 라운드 번호, 체인 식별자, 기법 번호, 채널, 성공 여부, 시작과 끝 시각, 컨테이너 번호다.

DB 에 안 들어가는 것이 하나 있다. 단계마다의 종료 코드와 출력은 화면에만 나오고 저장되지 않는다. 어느 명령이 왜 실패했는지는 나중에 못 본다. 성공 여부는 전체가 참인지 거짓인지 하나만 남는다.

7.5 탐지

Tetragon 이 내보내는 이벤트를 Mapper 가 받아 규칙 이름을 기법 번호로 바꿔 DB 에 넣는다. 이 부분은 내 담당이고 3장과 4장에서 다뤘다.

이벤트를 옮기는 방법은 메시지 큐가 아니라 유닉스 파이프다. `tetra getevents | mapper` 한 줄이다.

7.6 채점 경로가 둘이다

이게 지금 구조에서 제일 헷갈리는 부분이다. 같은 것을 재는 코드가 두 벌 있다.

경로 A Postgres 안에서 뷰로 끝낸다 `db/views.sql`
 데모 스크립트와 Grafana 대시보드가 이걸 읽는다
 오탐 측정이 여기에만 있다

경로 B Iceberg 로 옮긴 뒤 DuckDB 에서 계산한다 `dbt/models/marts/coverage.sql`
 Dagster 파이프라인이 이걸 돌린다
 오탐 측정이 여기에는 없다

Iceberg 는 파일로 데이터를 쌓아두는 방식이고, DuckDB 는 서버를 안 띄우고 파일 하나로 도는 분석용 데이터베이스다. dbt 는 SQL 로 쓴 변환 단계를 순서대로 돌려 지표 표를 만드는 도구다.

경로가 갈린 이유는 기록 위치 때문이다. Mapper 는 Postgres 에 쓰는데 dbt 쪽은 Iceberg 에서 읽는다. 그래서 데모에서는 Postgres 안에서 뷰로 끝내는 쪽을 쓴다.

문제가 하나 있다. 두 경로가 만드는 결과물 이름이 coverage 로 같다. 경로 A 는 뷰로 만들고 경로 B 는 테이블로 덮어쓴다. 같은 데이터베이스에서 둘을 다 돌리면 서로 지운다. 지금은 데모와 Dagster 를 같이 안 돌려서 안 부딪히고 있을 뿐이다.

그리고 이번에 내가 고친 채점 로직은 전부 경로 A 쪽이다. 오탐 정의, 단위 통일, 라운드 분리 셋 다 경로 B 에는 안 들어갔다.

7.7 되먹임

없다.

놓친 기법 목록은 화면에 출력만 되고 어떤 파일에도 큐에도 DB 에도 안 쓰인다. 생성 함수는 기법 번호 하나만 받는 구조라 놓친 목록을 받을 자리 자체가 없다. Dagster 는 대상 기법을 환경변수 상수로 고정해두고 그대로 넘긴다.

지금은 내가 화면을 보고 손으로 다음에 쓸 규칙을 정한다.

7.8 오케스트레이션

Dagster 가 asset 일곱 개를 순서대로 잇는다. asset 은 "이걸 만들어라" 단위 하나를 뜻한다.

round_id	이번 라운드 번호를 정한다
run_attack_round	VM 안에서 공격을 실행한다
collect_detections	탐지 이벤트를 모아 DB 에 넣는다
run_benign_round	정상 워크로드를 돌려 오탐을 쟀다
sync_iceberg	탐지 기록을 Iceberg 로 옮긴다
run_coverage_dbt	커버리지 지표를 다시 계산한다
test_coverage_dbt	지표가 말이 되는지 검사한다

들여쓰 만큼 앞의 것이 끝나야 뒤의 것이 돈다. run_benign_round 와 sync_iceberg 는 같은 깊이라 서로 순서가 없다.

round_id 와 run_benign_round 는 이번에 내가 넣었다. 라운드 번호를 안 넘기면 매번 1라운드에 쌓여 채점이 겹치는 문제가 있었고, 정상 워크로드가 파이프라인 안에 없어서 정밀도가 안 나왔다.

run_benign_round 를 collect_detections 뒤에 둔 이유는 Mapper 가 겹치지 않게 하려는 것이다. 공격 수집과 정상 수집이 동시에 돌면 같은 이벤트를 둘이 기록한다.

스케줄도 센서도 없다. 전부 사람이 눌러야 돈다.

주의할 점이 둘 있다. 첫째, Dagster 경로는 씨앗 체인을 안 쓰고 생성 경로를 탄다. 즉 데모에서 검증된 것과 다른 길이다. 둘째, 실행 환경이 셋으로 갈린다. 공격 실행과 탐지 수집은 Lima VM 안에서, Iceberg 와 dbt 는 맥에서, 정상 워크로드는 맥에서 VM 도커에 붙어서 돈다.

이 레포의 가상환경에 Dagster 와 dbt 가 안 깔려 있어서 내 컴퓨터에서는 이 파이프라인을 실제로 돌려보지 못했다. asset 코드는 맞춰뒀지만 실행 검증은 안 됐다.

7.9 인프라

compose 로 뜨는 것은 셋이다.

postgres 실행 기록, 탐지 기록, 정상 워크로드 기록
 neo4j ATT&CK 기법 정보
 grafana 커버리지 대시보드

Ollama 와 Tetragon 은 compose 에 없다. Ollama 는 맥에서 따로 띄우고 Tetragon 은 VM 안 도커로 띄운다.

Grafana 대시보드는 패널이 다섯이다. 탐지 성공 수, 놓친 수, 측정 불가 수, 기법별 커버리지 표, 라운드별 재현율 추이다. 전부 경로 A 의 뷰를 읽는다. 이번에 만든 정밀도와 F1 은 아직 대시보드에 안 올라가 있다.

8. 이름만 있고 안 도는 것

문서나 다이어그램에는 나오는데 코드에는 없는 것들이다. 발표나 보고에서 말할 때 조심해야 한다.

항목	상태
Redpanda 메시지 큐	없음. 디렉터리는 비어 있고 의존성도 없다. 실제로는 유닉스 파이프를 쓴다
Redis 잡큐	없음. 기술 스택 표에만 있다
MLflow	없음. 기술 스택 표와 아키텍처 그림에만 있다
Slack 사람 승인	절반. 알림과 버튼은 있고 버튼을 받는 서버가 없다
놓친 기법 되먹임	없음. 화면 출력만 있다
지식그래프의 공격 선후관계	없음. 전술 소속과 상하위 관계 둘뿐이다
io_uring 채널 기록	없음. 실행기와 Mapper 둘 다 채널 값이 syscall 로 박혀 있다
희수님의 3단계 검증기	코드는 있고 파이프라인에 안 붙어 있다. 지금은 실행도 안 된다
난수 표식 조인 키	코드는 있고 실행기가 안 쓴다. 조인은 여전히 컨테이너 번호와 시간창이다
CI, terraform, 설계 결정 기록	전부 빈 디렉터리다

io_uring 은 나눠 봐야 한다. 탐지 쪽 대조실험은 실제로 있다. 일부러 뚫리게 만든 대조군 규칙 넷, 검증용 C 프로그램, 데모 스크립트, 실측 결과표가 다 있다. 없는 것은 그 결과를 DB 에서 계산하는 부분이다. 실행기와 Mapper 가 채널 값을 syscall 로 고정해두고 있어서, 같은 공격을 두 경로로 싸도 DB 에는 구분이 안 남는다. 그래서 대조실험 결과를 지금은 손으로 세고 있다.

9. 남은 일

내 영역

오탐 시나리오가 규칙마다 고르지 않다. 마운트나 네임스페이스를 건드리는 정상 워크로드가 하나도 없어서 컨테이너 탈출 규칙 둘은 오탐을 영영 못 본다. 새로 만든 스트림 규칙 셋도 대응하는 정상 워크로드가 없다.

남은 오탐 둘은 규칙을 좁혀서 없앨 수 있는지 아직 모른다. 임시 경로 실행과 종료 신호 전송은 정상 동작과 공격이 실제로 겹친다. 좁히려면 지금 혹은 후에 없는 조건이 필요하다.

경로 B의 채점 로직에 이번 수정이 안 들어갔다. 오탐 정의, 단위 통일, 라운드 분리 셋 다 경로 A에만 있다.

팀 차원에서 정해야 할 것

앞 문서 5.7에 적어둔 논의 항목들이 대부분 그대로 남아 있다. 그중 이번 작업으로 사정이 바뀐 것만 적는다.

채널 기록 주체는 결론이 명확해졌다. 채널은 공격을 쏘는 쪽이 아는 정보이고, 실행 기록과 탐지 기록 두 테이블에 담을 칸이 이미 있다. 남은 일은 하드코딩 두 줄을 지우는 것뿐이다. 이게 안 되면 `io_uring` 대조실험이 이 프로젝트의 대표 결과물인데 지표 파이프라인에 안 실린다.

채점 경로 이중화는 새로 생긴 문제다. 같은 이름의 결과물을 두 경로가 서로 덮어쓴다. 어느 쪽을 정본으로 할지 정해야 한다. 오탐 측정이 경로 A에만 있으므로 지금은 A가 앞서 있다.

끊긴 고리 둘은 그대로다. 사람 승인과 되먹임이다. 이 둘이 이어져야 문서에 그린 6단계 순환이 코드로 완성된다.